作业 HW4实验报告 姓名：王尹囿 学号：2552304 日期：2026年4月18日

1. 涉及数据结构和相关背景

本次实验主要用到了数组、二叉搜索树、树结构以及映射表等数据结构，并结合排序、递归、父指针和层次信息处理输入。

第一题侧重于排序和贪心思想。通过将数组从小到大排序，优先使用较小元素，可以在给定和的限制下构造出更长的子序列。

第二题使用二叉搜索树维护整数集合。为了支持重复元素，结点中额外记录出现次数；为了方便删除和重连子树，结点还保存父指针。

第三题是家谱关系判断问题，需要把缩进文本还原成树结构。为了快速查找名字并判断亲属关系，程序使用了父指针和 `map<string, People_node*>` 来保存所有结点。

2. 实验内容

2.1 和有限的最长子序列

2.1.1 问题描述

给定整数数组 `nums` 和查询数组 `queries`，对每个查询值 `queries[i]`，求出 `nums` 中元素之和不超过 `queries[i]` 的子序列的最大长度。

2.1.2 基本要求

题目要求对每个查询都输出一个结果。数据范围不大，`n` 和 `m` 最大为 1000，因此可以采用较直接的实现方式完成计算。

2.1.3 数据结构设计

程序使用两个 `vector<int>` 分别保存 `nums` 和 `queries`。

实现步骤如下：

1. 将 `nums` 按升序排序。
2. 对每个查询值，从小到大遍历排序后的数组。
3. 只要当前元素不超过剩余预算，就把它加入子序列，并更新剩余值。

这样做的原因是：想让子序列长度尽可能长，优先选择较小的数一定更有利。

2.1.4 功能说明（函数、类）

`vector<int> split(const string &str)`：把一行输入按空格拆分为整数序列，返回 `vector<int>`。

`main()`：读取数组长度与查询长度，分别读入两行整数，完成排序、逐个查询和输出结果。

2.1.5 调试分析

这一题的关键点在于输入处理。程序先用 `cin` 读长度，再用 `getline` 读整行数组，因此需要先吞掉换行符，否则第一次 `getline` 会读到空串。

另一个容易出错的地方是贪心顺序。必须先排序再从小到大尝试，否则会错误地消耗预算，导致子序列长度变短。

由于数据范围只有 1000，当前实现的双重循环可以通过。如果继续优化，可以用前缀和加二分查找把每个查询降到对数级。

2.1.6 总结和体会

这道题的核心不是复杂的数据结构，而是正确选择贪心策略。排序后优先取小数，可以把“最多能取多少个”转化成一个简单的顺序扫描问题。

通过这题，我加深了对“局部最优能导出全局最优”这一类题目的理解，也熟悉了 `getline` 与 `cin` 混合输入时的处理方式。

2.2 二叉排序树

2.2.1 问题描述

题目要求实现一个支持重复关键字的二叉排序树，并完成插入、删除、查询元请参照报告模板.md的格式以及4道题目的要求以及我完成的cpp文件创作实验报告。素个数、查询最小值以及查询前驱等操作。

2.2.2 基本要求

BST 需要支持以下操作：

1. 插入一个整数。
2. 删除一个整数，若不存在则输出 `None`。
3. 查询某个整数出现了多少次。
4. 查询当前集合中的最小值。
5. 查询某个数字的前驱。

2.2.3 数据结构设计

程序定义了 `Tree_node` 结点结构，包含以下信息：

1. `value`：当前结点保存的关键字。
2. `number`：该关键字出现的次数。
3. `parent`：父结点指针。
4. `l_child` 和 `r_child`：左右孩子指针。

另外定义了 `Tree` 类来封装树的操作。类中保存根结点 `root`，并用一个结点列表记录已经分配的结点，便于析构时统一释放内存。

2.2.4 功能说明（函数、类）

`Tree_node`：BST 的基本结点类型，负责保存关键字、出现次数和链接关系。

`Tree::find(int num, Tree_node *node_p)`：在以 `node_p` 为根的子树中查找值为 `num` 的结点。

`Tree::start_find(int num)`：从根结点开始查找，并返回该值的出现次数；如果不存在则返回 0。

`Tree::start_insert(int num)` 和 `Tree::insert(int num, Tree_node *node_p)`：在树中插入一个整数。若结点已存在，则直接增加计数。

`Tree::find_min(Tree_node *start)`：返回以 `start` 为根子树中的最小结点。

`Tree::find_max(Tree_node *start)`：返回以 `start` 为根子树中的最大结点。

`Tree::delete_node(int num)`：删除一个整数。若结点出现次数大于 1，则只减少计数；若结点只有一个孩子或没有孩子，则直接重连；若左右孩子都存在，则用中序后继替换。

`Tree::get_min()`：返回当前树中最小值。

`Tree::get_predecessor(int x)`：从根结点向下搜索，记录所有小于 `x` 的最大值，作为前驱返回。

`main()`：按输入的操作编号分发到对应函数，完成所有查询和输出。

2.2.5 调试分析

这一题最容易出错的是删除操作。BST 删除时需要同时考虑四种情况：不存在、重复次数大于 1、只有一个孩子、左右孩子都存在。若忽略其中任意一种，树结构都会被破坏。

另一个重点是重复关键字。程序不能简单地把同一个值插入成多个结点，而是应该在结点中维护出现次数，这样查询和删除都会更稳定。

前驱查询也需要特别处理边界情况。如果集合中不存在比 `x` 更小的元素，则输出 `None`。

2.2.6 总结和体会

BST 的实现比顺序结构更复杂，但它把查找、插入和删除都建立在同一套树形逻辑上。通过这题，我更加熟悉了父指针在树结构维护中的作用，也更清楚如何处理重复元素和中序后继替换。

2.3 家谱关系判断

2.3.1 问题描述

题目给出一棵用缩进文本表示的家谱树，以及若干关系陈述，需要判断每条陈述是否正确。支持的关系包括 child、parent、sibling、descendant 和 ancestor。

2.3.2 基本要求

输入由多个测试用例组成，最后以 `0 0` 结束。每个测试用例中，先读入家谱树，再读入若干条关系查询，最后对每条查询输出 `True` 或 `False`，并在每组测试用例后输出一个空行。

2.3.3 数据结构设计

程序定义了 `People_node` 结构：

1. `name`：人名。
2. `parent`：父结点指针。
3. `children_list`：孩子结点集合，使用 `map<string, People_node*>` 保存。

同时定义了 `Family` 类：

1. `root` 保存家谱根结点。
2. `people_node_list` 记录所有人名到结点的映射，用于快速查找。

构建树时，程序使用 `parent_stack` 记录每一层最近出现的结点。通过统计行首空格数量，可以得到当前结点的层次，从而找到它的父结点。

2.3.4 功能说明（函数、类）

`int count_leading_spaces(const string &str)`：统计字符串开头空格数量，用来判断缩进层次。

`Family::Family(string name)`：初始化家谱，创建根结点并加入映射表。

`Family::~Family()`：释放家谱中所有结点，避免内存泄漏。

`Family::add_child(string parent_name, string child_name)`：在指定父结点下添加孩子。

`Family::is_child(string child_name, string parent_name)`：判断 `child_name` 是否是 `parent_name` 的孩子。

`Family::is_parent(string parent_name, string child_name)`：判断 `parent_name` 是否是 `child_name` 的父母。

`Family::is_sibling(string name1, string name2)`：判断两个人是否有相同父结点。

`Family::is_ancestor(string anc_name, string desc_name)`：沿父指针向上搜索，判断是否为祖先；同一个人也视为自己的祖先。

`Family::is_descendant(string desc_name, string anc_name)`：直接复用祖先判断逻辑。

`main()`：读取每个测试用例的家谱和查询语句，解析句子格式，输出判断结果，并处理多组测试数据。

2.3.5 调试分析

这一题的输入格式比较复杂，难点在于把缩进文本正确还原成树。程序先统计每行前导空格，再结合 `parent_stack` 找到当前节点的父结点，这是构造家谱树的关键。

查询语句也需要注意格式解析。每句都以句号结尾，因此在读出最后一个单词后要去掉末尾的 `.`，否则名字会匹配失败。

此外，题目规定祖先、后代和兄弟关系允许是自己，因此判断逻辑必须专门处理自反情况。程序在祖先判断中先检查名字是否相同，再沿父指针向上查找。

2.3.6 总结和体会

这道题体现了“文本解析 + 树结构建模 + 关系查询”的综合能力。将缩进信息转换成树后，原本看起来复杂的家谱问题就能变成简单的父指针遍历问题。

通过本题，我更加熟悉了如何把输入中的隐含结构抽象为树，也体会到映射表在快速定位结点时的实用性。

3. 实验总结

本次实验的三道题分别对应了贪心、树结构维护和树关系判断三个方向。第一题让我体会到排序和贪心结合的简洁性；第二题让我熟悉了 BST 中删除、前驱和重复值的处理；第三题则训练了我把文本输入转化为树模型的能力。

总体来看，这三题都强调了数据结构选择的重要性。只有先把问题中的“关系”抽象清楚，后续实现才会稳定。今后在做类似题目时，我会更加注意输入格式、边界条件以及重复元素等细节，尽量在建模阶段就把潜在问题处理掉。